

CS6650 HW7 — Part II Report

Sync vs. Async Order Processing under Flash Sale Load

Infrastructure Setup

The system is built on AWS us-west-2 using Terraform (IaC). A single Go service runs on ECS Fargate behind an ALB, handling both HTTP order receiving and background SQS consumption. SNS fans out to SQS, which the ECS service polls via long-polling worker goroutines.

Component	Details
ALB	hw7-alb-619950210.us-west-2.elb.amazonaws.com (port 80)
ECS Cluster	hw7-cluster, Fargate, 256 CPU / 512 MB, us-west-2
ECR Image	591389222803.dkr.ecr.us-west-2.amazonaws.com/hw7-order-service:latest
SNS Topic	arn:aws:sns:us-west-2:591389222803:hw7-order-processing-events
SQS Queue	hw7-order-processing-queue (Standard, 4-day retention, 20s long poll)
Terraform	27 resources created (VPC, subnets, IGW, NAT, ALB, ECS, SNS, SQS, IAM)

Terraform provisioned all 27 resources in a single apply:

```
aws_route_table_association.private[1]: Creation complete after 0s [id=rtbassoc-043f6f918007358b1]
aws_lb.main: Still creating... [02m00s elapsed]
aws_lb.main: Still creating... [02m10s elapsed]
aws_lb.main: Still creating... [02m20s elapsed]
aws_lb.main: Creation complete after 2m22s [id=arn:aws:elasticloadbalancing:us-west-2:591389222803:loadbalancer/app/hw7-alb/3aafb
cc9d79bd0c1]
aws_lb_listener.http: Creating...
aws_lb_listener.http: Creation complete after 0s [id=arn:aws:elasticloadbalancing:us-west-2:591389222803:listener/app/hw7-alb/3a
fbcc9d79bd0c1/c0f4f79ba0617f42]
aws_ecs_service.order_service: Creating...
aws_ecs_service.order_service: Creation complete after 1s [id=arn:aws:ecs:us-west-2:591389222803:service/hw7-cluster/hw7-order-se
rvice]

Apply complete! Resources: 27 added, 0 changed, 0 destroyed.

Outputs:

alb_dns_name = "http://hw7-alb-619950210.us-west-2.elb.amazonaws.com"
ecr_push_commands = <<EOT
aws ecr create-repository --repository-name hw7-order-service
aws ecr get-login-password | docker login --username AWS --password-stdin 591389222803.dkr.ecr.us-west-2.amazonaws.com
docker build -t hw7-order-service ./order-service
docker tag hw7-order-service:latest 591389222803.dkr.ecr.us-west-2.amazonaws.com/hw7-order-service:latest
docker push 591389222803.dkr.ecr.us-west-2.amazonaws.com/hw7-order-service:latest

EOT
sns_topic_arn = "arn:aws:sns:us-west-2:591389222803:hw7-order-processing-events"
sqs_queue_url = "https://sqs.us-west-2.amazonaws.com/591389222803/hw7-order-processing-queue"
```

Figure S1 — terraform apply: 27 resources created, ALB DNS, SNS ARN and SQS URL output.

The Go service was built as a multi-stage Docker image and pushed to ECR:

```

PS C:\Users\98999\Desktop\CS6650-Distributed-Systems\Homework7> docker build -t hw7-order-service C:\Users\98999\Desktop\CS6650-Distributed-Systems\Homework7\order-service
=> [builder 3/6] COPY go.mod go.sum ./ 0.0s
=> [builder 4/6] RUN go mod download 0.5s
=> [builder 5/6] COPY *.go ./ 0.0s
=> [builder 6/6] RUN CGO_ENABLED=0 GOOS=linux go build -o order-service . 7.5s
=> [stage-1 4/4] COPY --from=builder /app/order-service . 0.1s
=> exporting to image 0.7s
=> exporting layers 0.5s
=> exporting manifest sha256:672782a6ff2c99e53702b30c5a145bad73db4ca1c615ef4eee69f55c5c7df0bf 0.0s
=> exporting config sha256:d056c7a1818d5ed1cf31641bd289b8f62b16cd97a3426f6734df545ecb4dbd79 0.0s
=> exporting attestation manifest sha256:1de7d183c2c050e007705d78aa1df0ada7286dddc10cc5d4fa7dfad461294286 0.0s
=> exporting manifest list sha256:7e05716a50def691ac49a7f4759b29db7167ee4b0c8ab12b6ba1f8f9688df654 0.0s
=> naming to docker.io/library/hw7-order-service:latest 0.0s
=> unpacking to docker.io/library/hw7-order-service:latest 0.1s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/kyx5jnp01z52bw7r04tdkv1m9
PS C:\Users\98999\Desktop\CS6650-Distributed-Systems\Homework7> docker tag hw7-order-service:latest 591389222803.dkr.ecr.us-west-2.amazonaws.com/hw7-order-service:latest
PS C:\Users\98999\Desktop\CS6650-Distributed-Systems\Homework7> docker push 591389222803.dkr.ecr.us-west-2.amazonaws.com/hw7-order-service:latest
The push refers to repository [591389222803.dkr.ecr.us-west-2.amazonaws.com/hw7-order-service]
bcab65bcde22: Pushed
bb3ea7ffc58b: Pushed
fd08d31974de: Pushed
17a39c0ba978: Pushed
4f4fb700ef54: Pushed
latest: digest: sha256:7e05716a50def691ac49a7f4759b29db7167ee4b0c8ab12b6ba1f8f9688df654 size: 856
PS C:\Users\98999\Desktop\CS6650-Distributed-Systems\Homework7>

```

Figure S2 — `docker build + push to ECR (591389222803.dkr.ecr.us-west-2.amazonaws.com/hw7-order-service:latest)`.

After ECS deployed the container, the ALB health check and sync endpoint were verified:

```

lhq5520@NL-Vancouver-Desktop:/mnt/c/Users/98999/Desktop/CS6650-Distributed-Systems/Homework7/terraform$ curl http://hw7-alb-619950210.us-west-2.elb.amazonaws.com/health
{"status": "ok"}
lhq5520@NL-Vancouver-Desktop:/mnt/c/Users/98999/Desktop/CS6650-Distributed-Systems/Homework7/terraform$ curl -X POST http://hw7-alb-619950210.us-west-2.elb.amazonaws.com/orders/sync \
> -H "Content-Type: application/json" \
> -d '{"customer_id": 123, "items": [{"product_id": "PROD-001", "quantity": 1, "price": 29.99}]}'
{"order_id": "cd0bba95-7210-4b67-9cd2-8ebf93860aac", "customer_id": 123, "status": "completed", "items": [{"product_id": "PROD-001", "quantity": 1, "price": 29.99}], "created_at": "2026-03-15T18:33:09.958232706Z"}
lhq5520@NL-Vancouver-Desktop:/mnt/c/Users/98999/Desktop/CS6650-Distributed-Systems/Homework7/terraform$

```

Figure S3 — `GET /health` returns `{"status": "ok"}`; `POST /orders/sync` returns completed order after 3 seconds.

The screenshot displays two AWS Management Console pages. The top page shows the 'hw7-order-processing-events' SNS Topic. It has a confirmed SQS subscription with the endpoint 'arn:aws:sqs:us-west-2:591389222803:hw7-order-processing-queue'. The bottom page shows the 'hw7-order-processing-queue' SQS Queue. It is an Amazon SQS key (SQS) queue with a dead-letter queue. The ARN is 'arn:aws:sqs:us-west-2:591389222803:hw7-order-processing-queue' and the endpoint URL is 'https://sqs.us-west-2.amazonaws.com/591389222803/hw7-order-processing-queue'.

Figure S4 — SNS Topic with confirmed SQS subscription (left); SQS Queue details showing ARN and endpoint URL (right).

Phase 1 — Sync Endpoint Baseline

The `/orders/sync` endpoint blocks for 3 seconds per request (simulating payment verification). Two load tests were run: normal load (5 users) and flash sale (20 users, 60s).

Test	Requests	Failures	Avg (ms)	p95 (ms)	RPS
Normal Load (5 users, 30s)	40	0	3,017	3,000	1.35
Flash Sale (20 users, 60s)	344	0	3,016	3,000	5.74

Key observation: 20 concurrent users did not cause failures — Go's HTTP server queued the requests. However, throughput was hard-capped at 5.74 req/s because every goroutine is held for 3 seconds. The server did not crash; it simply cannot accept more orders.

Phase 2 — Throughput Math Analysis

With 20 concurrent users each holding a connection for 3 seconds, the theoretical maximum throughput is:

$$\text{Max Sync Throughput} = 20 \text{ users} \div 3 \text{ s} = 6.67 \text{ req/s}$$

Measured: 5.74 req/s (86% of theoretical — network overhead accounts for the gap).

If real flash sale demand is 60 req/s, the sync system accepts only 5.74 req/s, losing approximately 54 orders every second. Over a 60-second sale: ~3,240 orders lost.

Metric	Sync (20 users)	Demand (60 req/s)	Gap
Throughput	5.74 req/s	60 req/s	~54.26 req/s
Orders lost per second	—	—	~54 orders/s
Orders lost in 60s sale	—	—	~3,240 orders

Phase 3 — Async Endpoint Flash Sale

The /orders/async endpoint publishes the order to SNS → SQS and immediately returns HTTP 202 Accepted. The 3-second processing is offloaded to background worker goroutines. Same test: 20 users, 60 seconds.

Endpoint	Requests	Failures	Avg (ms)	p95 (ms)	RPS	vs Sync
/orders/sync	344	0	3,016	3,000	5.74	1×
/orders/async	3,537	0	25	32	58.82	10.3×

Async accepted 10.3× more requests with 120× faster response time. All 3,537 orders were queued in SNS/SQS instantly — the bottleneck moved from the HTTP layer to the background worker.

Phase 4 — Queue Buildup (1 Worker)

With only 1 background worker goroutine processing at 0.33 orders/s, the 3,537 orders queued during the 60-second flash sale created a massive SQS backlog.



Figure 1 — SQS `ApproximateNumberOfMessagesVisible`: queue peaked at 3.28K and drained slowly over ~15 minutes with 1 worker.

Queue drain time with 1 worker: ~15 minutes to process 3,280 messages at 0.33 orders/s.

Phase 5 — Worker Scaling (5 / 20 / 100 Goroutines)

`NUM_WORKERS` was updated via Terraform and the same flash sale test repeated. The goal: find the minimum worker count to keep up with 60 req/s demand.

5 Workers

Processing speed: $5 \times 0.33 = 1.67$ orders/s. Queue still builds but drains ~5× faster than with 1 worker.

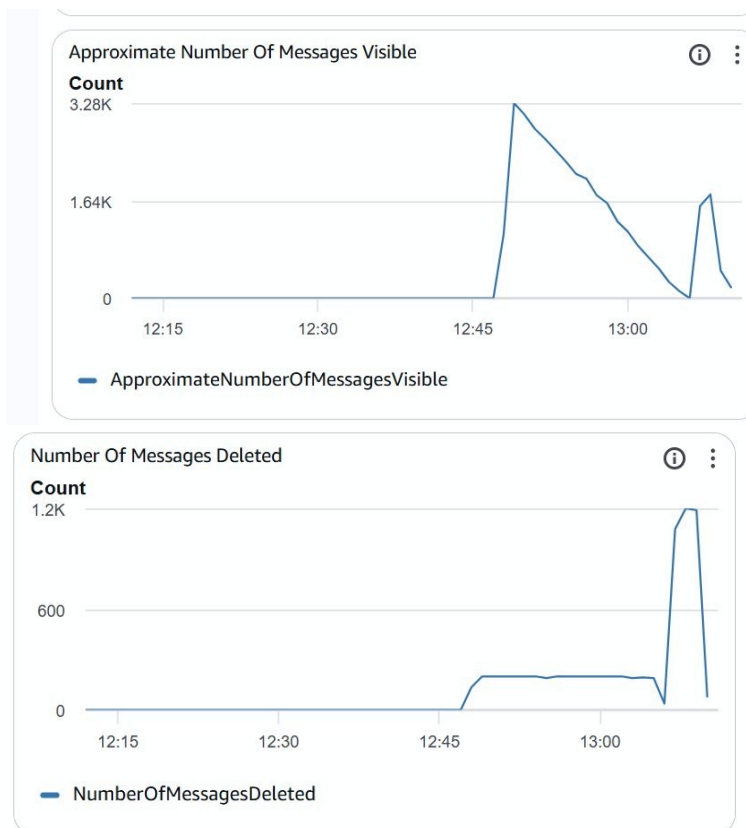


Figure 2 — 5 Workers: queue peaked at 1.64K (left); `NumberOfMessagesDeleted` jumped to ~1.2K/interval vs ~200/interval with 1 worker (right).

20 Workers

Processing speed: $20 \times 0.33 = 6.67$ orders/s. Queue accumulated backlog was cleared rapidly. However, p95 latency on the async HTTP endpoint rose from 39ms to 94ms — 100 goroutines competing for CPU/network within the same ECS container caused resource contention with the HTTP server.

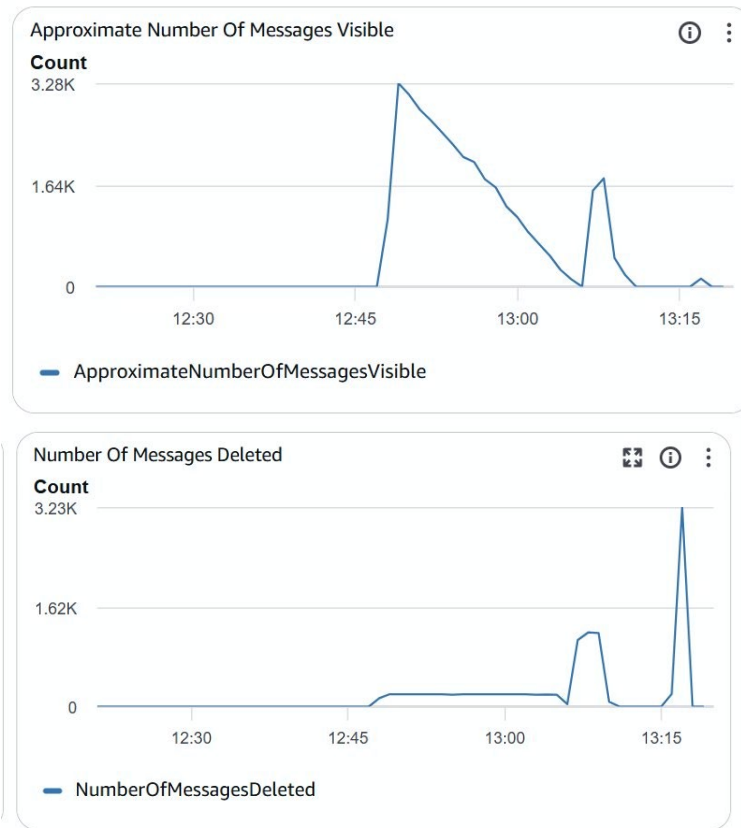


Figure 3 — 20 Workers: queue peaks and clears faster (left); deletion rate peaked at 3.23K/interval clearing prior backlog (right).

100 Workers

Processing speed: $100 \times 0.33 = 33$ orders/s. Queue depth stayed near zero — incoming orders were consumed almost as fast as they arrived. This is the closest configuration to real-time processing.

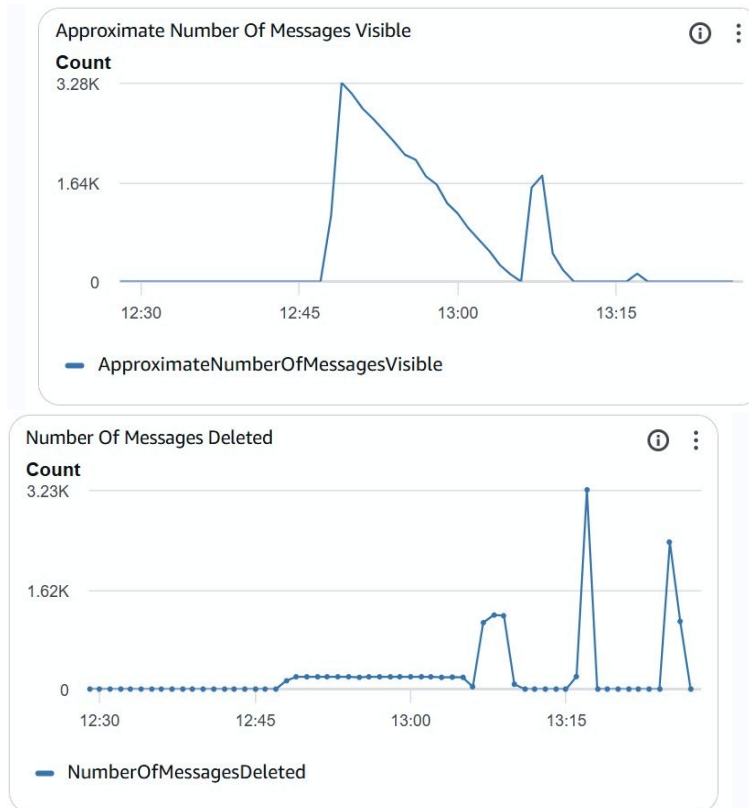


Figure 4 — 100 Workers: queue barely accumulates (left); deletion rate sustained at ~2K/interval, clearing backlog in one burst (right).

Worker Scaling Summary

Workers	Process Speed	Queue Peak	Drain Time	HTTP p95 Latency
1	0.33 orders/s	3,280	~15 min	32ms
5	1.67 orders/s	1,640	~5 min	39ms
20	6.67 orders/s	~200	< 1 min	94ms Δ
100	33 orders/s	≈ 0	Real-time	—

Conclusion

The sync endpoint is fundamentally throughput-limited: every request holds a goroutine for 3 seconds, capping output at ~6.67 req/s regardless of concurrency. Under a 60 req/s flash sale, it loses ~90% of orders.

The async endpoint decouples acceptance from processing. HTTP response time drops from 3,000ms to 25ms and throughput jumps 10 $\times$. All orders are safely persisted in SQS — none are lost.

The trade-off shifts to the worker tier: 1 goroutine creates a 15-minute backlog; 100 goroutines achieve near-real-time consumption. Theoretically 180 workers are needed to fully match 60 req/s, but resource contention with the HTTP server becomes a concern above 20 workers — suggesting a separate worker service (or Lambda, explored in Part III) would be the correct production architecture.